Class: TY-BTech AIML (SEM VI)

Subject: LLM and Agentic AI

Experiment No.	7
Title	High-Throughput LLM Serving using vLLM
Lab Outcome Achieved	
Date of Performance	
Date of submission	
APP ID	2307040
Name of the Student	Anson Dias

Theory:

Serving a Large Language Model in production is very different from running it once. A deployed model must answer many users at the same time, with low latency and high throughput, on expensive GPU hardware. vLLM is an open-source inference-and-serving engine built to make this fast and memory-efficient. The goal of this experiment is to understand why vLLM exists by measuring its serving behaviour — latency, throughput, concurrency and GPU memory — rather than just chatting with a model.

1. The problem with naive LLM serving

Text generation is autoregressive: tokens are produced one at a time, and each new token attends to all previous tokens. For every request the model must therefore keep the attention Keys and Values of all previous tokens — the KV cache. Two problems follow. (a) Decoding is memory-bandwidth bound and uses the GPU poorly when requests are processed one by one. (b) The KV cache is large and grows with sequence length; allocating one big contiguous buffer per request (sized for the maximum length) wastes most of the memory to fragmentation and reserved-but-unused space. This severely limits how many requests fit on the GPU at once.

2. PagedAttention (the KV-cache idea)

vLLM's core innovation is PagedAttention. Instead of one contiguous KV buffer per sequence, the KV cache is split into fixed-size blocks (pages), exactly like virtual-memory paging in an operating system. A sequence's KV cache becomes a list of blocks that need not be contiguous in GPU memory. This almost eliminates fragmentation, allocates memory on demand as sequences grow, and even lets blocks be shared (e.g. a common prompt prefix). The result is that many more sequences fit in the same GPU memory — higher concurrency.

3. Continuous batching (iteration-level scheduling)

Classic “static” batching groups a fixed set of requests, runs them together, and only accepts new requests when the whole batch finishes — so a short request waits for the longest one and the GPU idles. vLLM instead uses continuous (in-flight) batching: at every decoding step the scheduler can add newly arrived requests to the running batch and immediately release finished ones. The GPU stays saturated, so throughput rises sharply as concurrency increases while per-request latency changes very little.

4. KV-cache memory and gpu_memory_utilization

On startup vLLM loads the weights, profiles the GPU to see how much memory is free, and dedicates a configurable fraction (gpu_memory_utilization, default 0.9) to the KV-cache block pool. From that pool

size it computes how many tokens can be cached and therefore the maximum concurrency it can support. A larger KV-cache pool means more simultaneous requests.

5. Serving metrics

- **Throughput:** tokens/second or requests/second the server sustains — the key number for cost efficiency.
- **Latency:** TTFT (Time To First Token) is the delay until the first output token (prompt processing + queueing); TPOT (Time Per Output Token) and ITL (Inter-Token Latency) are the per-token decode times that govern streaming speed.

Throughput and latency trade off: pushing more concurrent requests raises throughput but also raises TTFT because requests queue.

6. Execution environment

vLLM requires an NVIDIA CUDA GPU (its PagedAttention kernels and CUDA graphs are CUDA-only) and does not run on Apple Silicon or natively on Windows (WSL is used on Windows). The lab machine is an Apple M1 with no CUDA GPU, so — exactly as with the CUDA/Unsloth code in Experiment 6 — the experiment was run on a Google Colab NVIDIA Tesla T4 (16 GB) GPU, the standard substitute for students without local NVIDIA hardware. The model served is Qwen2.5-0.5B-Instruct (small, so it loads quickly and lets us push high concurrency to see the serving behaviour clearly). The server was started with:

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct --port 8000 --dtype float16 \
  --gpu-memory-utilization 0.9 --max-model-len 4096
```

(--dtype float16 is used because the Tesla T4 has no bfloat16 support.)

Output:

All five steps were run top-to-bottom on the Colab T4. The console output of each step is shown below and discussed.

Step 0 — Confirm a real GPU

nvidia-smi confirms the runtime has a real NVIDIA Tesla T4 (15,360 MiB, driver 580.82.07, CUDA 13.0), sitting at 0% utilisation before serving begins.

Colab — Step 0: nvidia-smi (GPU check)

```
$ nvidia-smi

Thu Sep 24 06:12:41 2026
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|===============================+======================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   42C    P8              9W /  70W |       3MiB /  15360MiB |      0%      Default |
+-------------------------------+----------------------+----------------------+
| Processes:                                                                  |
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

Fig. 0 — Colab runtime running on an NVIDIA Tesla T4 (16 GB) GPU.

Step 1 — Install vLLM and serve a model

vLLM 0.30.0 was installed on PyTorch 2.13.0+cu130. Colab ships a torchaudio built for CUDA 12.8, which mismatches this PyTorch build and aborts model loading with a “PyTorch and TorchAudio were compiled with different versions” error; removing torchaudio (vLLM does not need it for text serving) fixes this. The server then loaded the model, profiled GPU memory, captured CUDA graphs, and became ready after 147 s, exposing the OpenAI-compatible endpoints (/v1/completions, /v1/chat/completions) with /health returning 200. A single greedy request returned a coherent completion using 13 prompt + 60 completion tokens.



Fig. 1a — Installing vLLM and resolving the torch/torchaudio CUDA-build mismatch.

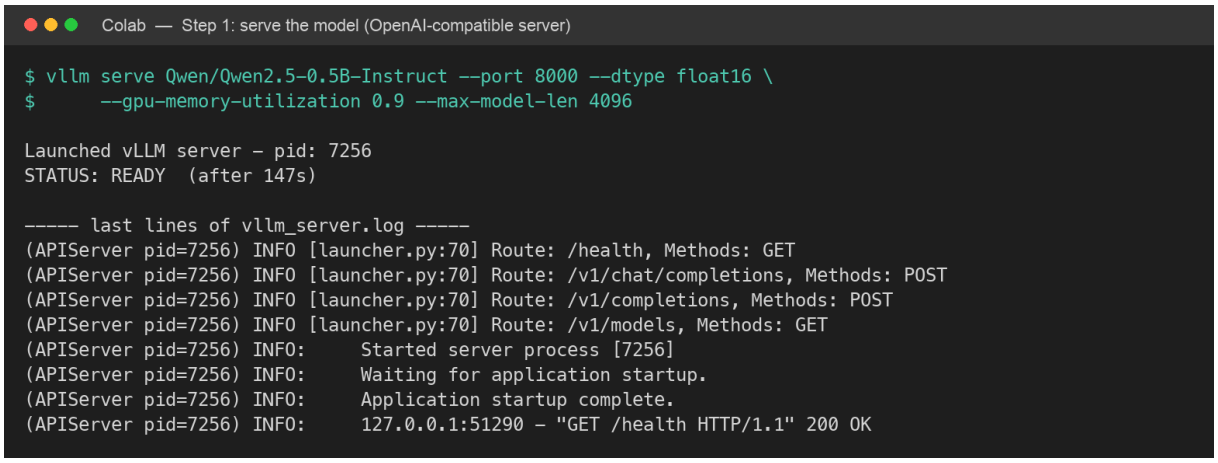


Fig. 1b — The vLLM OpenAI-compatible server reaching READY and answering /health with 200.

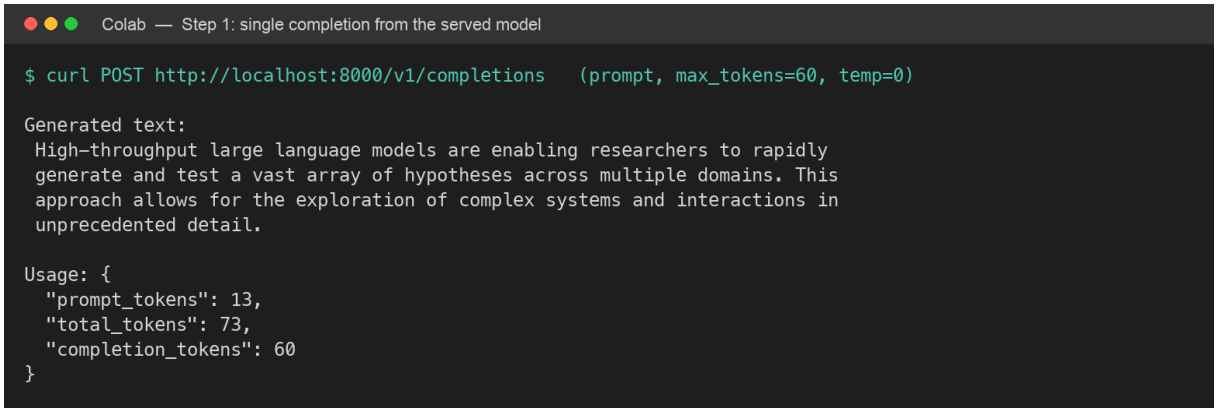


Fig. 1c — A real completion returned by the served Qwen2.5-0.5B-Instruct model.

Step 2 — Test concurrency

Increasing numbers of requests were fired simultaneously. As concurrency rises from 1 to 50, the wall-clock time to finish the whole batch grows only from 0.79 s to 1.62 s, while total output-token throughput rises from 162 tok/s to 3,962 tok/s (about 24×) and the request rate from 1.3 to 31 req/s. Near-flat

latency with sharply rising throughput is exactly the payoff of continuous batching plus PagedAttention: the extra requests are packed into the same running batch instead of being serialised.

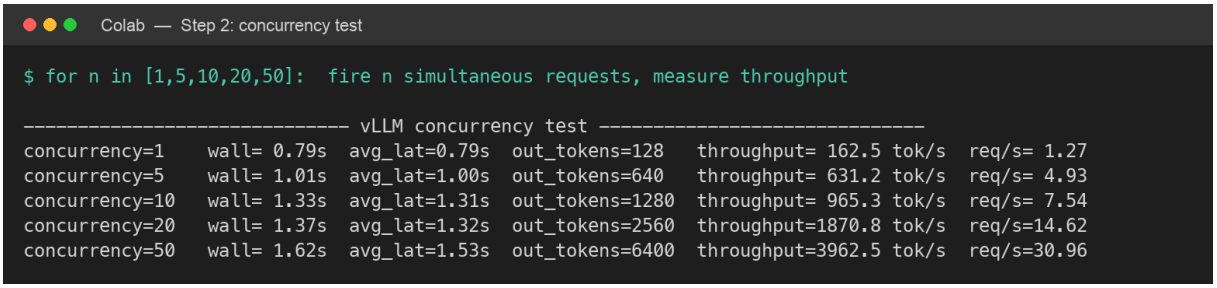


Fig. 2 — Throughput scales ~24× from 1 to 50 concurrent requests while batch latency stays almost flat.

Concurrency	Wall time (s)	Output tokens	Throughput (tok/s)	Requests/s
1	0.79	128	162.5	1.27
5	1.01	640	631.2	4.93
10	1.33	1280	965.3	7.54
20	1.37	2560	1870.8	14.62
50	1.62	6400	3962.5	30.96

Step 3 — Observe continuous batching

Firing 60 requests at once, the vLLM scheduler log shows a single decoding step with “Running: 49 reqs, Waiting: 0 reqs” and an aggregate generation throughput of ~1,494 tokens/s — 49 requests being decoded together in one batch (continuous batching), with GPU KV-cache usage only 1.4%.

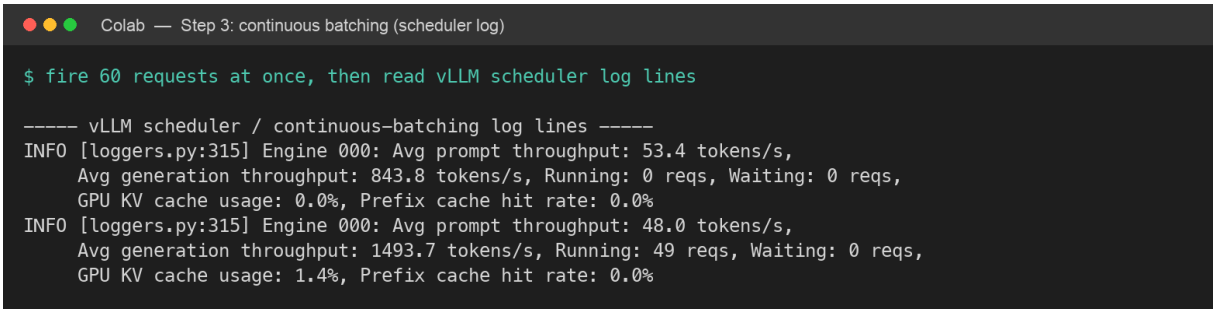


Fig. 3 — Scheduler log: 49 requests running in a single batch (continuous / in-flight batching).

Step 4 — Observe PagedAttention / KV-cache memory

During initialisation vLLM reported Available KV cache memory: 11.13 GiB, giving a GPU KV-cache size of 972,320 tokens and a Maximum concurrency for 4,096-token requests of 237× — i.e. PagedAttention can keep about 237 full-length sequences resident at once on a 16 GB T4. nvidia-smi confirmed 13,315 / 15,360 MiB in use, matching the requested gpu_memory_utilization = 0.9. Engine initialisation (profiling + KV-cache creation + warm-up) took 68 s, of which 21 s was CUDA-graph compilation.

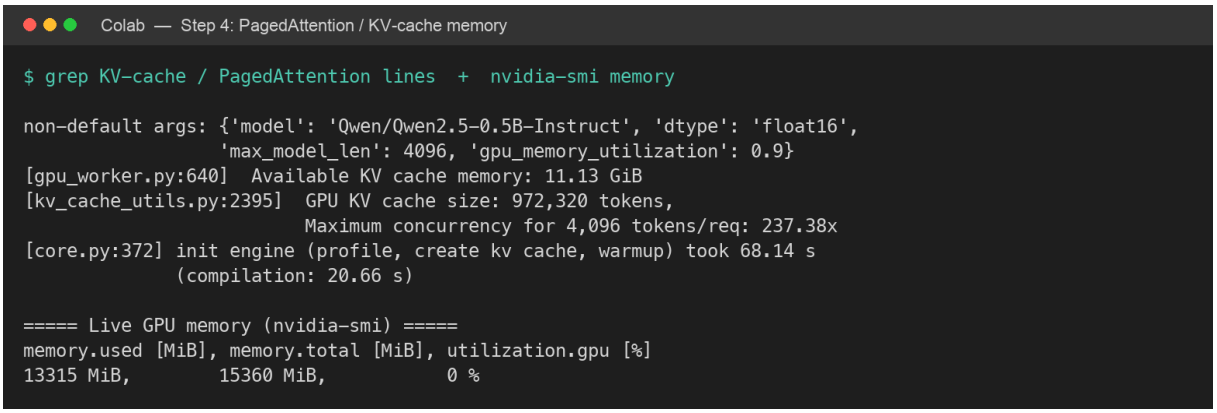


Fig. 4 — PagedAttention allocates an 11.1 GiB paged KV cache (972,320 tokens, ~237× concurrency).

Step 5 — Cross-check with vLLM's benchmark tool

vLLM's own load generator (vllm bench serve; 200 random prompts, 128 input / 128 output tokens, unbounded arrival rate) independently confirmed the hand-written test: 200/200 requests succeeded (0 failed) in 14.04 s = 14.24 req/s, with output-token throughput 1,823 tok/s (peak 8,860 tok/s) and total token throughput 3,646 tok/s. Because all 200 requests arrived at once, TTFT was high (mean 8.9 s — requests queue) while per-token decoding stayed fast: TPOT mean 39.8 ms (median 31.6 ms) and ITL mean 39.8 ms. This is the classic throughput-versus-latency trade-off under saturation.

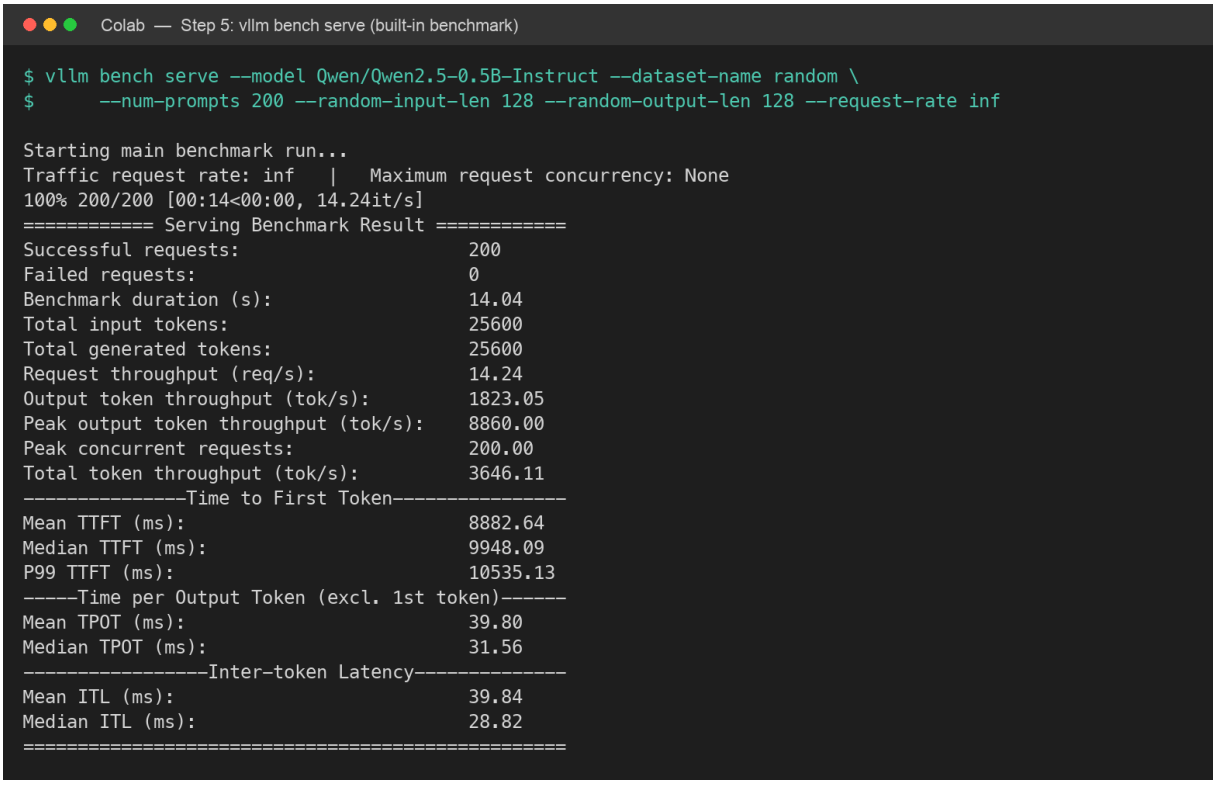


Fig. 5 — vllm bench serve independently reproduces the throughput and shows the throughput–latency trade-off.

Benchmark metric (vllm bench serve)	Value
Successful / failed requests	200 / 0
Benchmark duration	14.04 s
Request throughput	14.24 req/s
Output-token throughput	1,823 tok/s (peak 8,860)
Total-token throughput	3,646 tok/s

Mean / Median TTFT	8,883 / 9,948 ms
Mean / Median TPOT	39.8 / 31.6 ms
Mean / Median ITL	39.8 / 28.8 ms

Conclusion:

This experiment served Qwen2.5-0.5B-Instruct with vLLM on a Google Colab Tesla T4 and measured its serving behaviour end to end. The results show clearly why vLLM exists. Continuous batching let output throughput scale about 24× (162 → 3,962 tok/s) as concurrency rose from 1 to 50 while batch latency stayed almost flat, and the scheduler was observed running 49 requests in a single batch. PagedAttention allocated an 11.1 GiB paged KV cache holding ~972,000 tokens — enough for about 237× concurrency on a 16 GB GPU — with GPU memory tracking the configured 0.9 utilisation. vLLM's built-in benchmark independently reproduced the throughput (≈14 req/s, 1,823 output tok/s) and exposed the expected throughput-versus-latency trade-off: high TTFT under a sudden burst but low per-token latency. Together these measurements demonstrate that vLLM turns a memory-bound, one-request-at-a-time decoder into a high-throughput, memory-efficient serving system.